IT Italiano · GB **English**

💻 ichosynth — Development Environment

Complete, up-to-date guide for Windows and macOS

Everything you need to build, modify, and upload the firmware: from the toolchain to the libraries, all the way to your first upload onto the Teensy 4.1.



> ⚡ **In a hurry?** ichosynth runs the real **TŒRN** firmware, and one command builds it for our hardware: `python firmware/build_toern.py`. Jump to chapter 3. The rest of the guide explains every step and prerequisite.

📑 Table of Contents

- 1 · What we install
- 2 · Requirements
- 3 · The fast track: one command builds it all
- 4 · Installing the toolchain (arduino-cli)
- 5 · Adding Teensy support (the teensy:avr core)
- 6 · The libraries
- 7 · How the build script works
- 8 · Building and uploading
- 9 · Building from the Arduino IDE (alternative)
- 10 · Optional tools (Python, Git)
- 11 · Checklist and common problems

---

1 · What we install

ichosynth is a **Teensy 4.1** running the real **TŒRN** firmware (by SP_ / soundpauli — https://toern.live) on cheap, hand-soldered parts. We don't keep a separate copy of TŒRN in the repo: a build script fetches the upstream sources and compiles them for our hardware. Here's what you need:

Component	Version	What it's for	Required
arduino-cli	latest	the command-line build toolchain	✅
teensy:avr core	**≥ 1.61.0**	the Teensy compiler + Teensy Loader	✅
Python	3.x	runs `firmware/build_toern.py` (the build script)	✅
Git	latest	the script clones the TŒRN sources on first run	✅
Custom drivers	in the repo	`i2cEncoderLibV2`, `FastTouch`, `IchosOled` — already in `firmware/libraries/`	✅ (bundled)
GitHub CLI	latest	optional, for version-controlling the project	optional

> ⚙️ **The one thing that bites everyone:** TŒRN is a single ~23k-line translation unit, and the Teensy gcc **crashes at the default** `-O2` (exits `0xffffffff` — an internal compiler error / out-of-memory). The fix is to compile at `-O1` (`opt=o1std`). The build script sets this for you; if you build from the Arduino IDE instead, you must pick **Tools → Optimize → "Fast"**. See chapter 7.

---

2 · Requirements

- **Windows**: Windows 10 or 11, 64-bit.
- **macOS**: macOS 10.15 or later; native support for **Intel** and **Apple Silicon** (M1/M2/M3/M4).
- **A micro-USB data cable** (not charge-only) for the Teensy 4.1.
- A **Teensy 4.1 with 16 MB PSRAM** (both chips) soldered on — TŒRN needs the PSRAM for its sample buffers.
- ~1 GB of disk space for the toolchain + core + libraries.

---

3 · The fast track: one command builds it all

Once you have **arduino-cli**, the **teensy:avr core**, **Python**, and **Git** (chapters 4–5), building the firmware is a single command from the repo root:

```
python firmware/build_toern.py
```

This produces the firmware at `firmware/build/toern.hex`. To build *and* flash in one go (if `teensy_loader_cli` is on your PATH):

```
python firmware/build_toern.py --flash
```

Useful flags:

Flag	Effect
`--flash`	after building, upload with `teensy_loader_cli`
`--keep`	keep the temporary build folder (for inspecting the staged/patched sources)

> 💡 The **first** build is slow (the Teensy core is compiled once) and it also **clones the TŒRN sources** into `emulator/toern-src/` if they aren't there yet. Later builds are fast.
>
> 🖱️ Just want to flash a **prebuilt** `.hex`? There's a one-click GUI flasher in `tools/flasher/` — see chapter 8.3.

---

4 · Installing the toolchain (arduino-cli)

We build from the command line with `arduino-cli`.

Windows

```
winget install ArduinoSA.CLI
```

macOS

```
brew install arduino-cli
```

Check it's on your PATH:

```
arduino-cli version
```

> ℹ️ **Drivers**: on Windows 10/11 and on macOS **no additional drivers are needed** for the Teensy 4.1.

---

5 · Adding Teensy support (the teensy:avr core)

The Teensy compiler and Teensy Loader come from the **teensy:avr core**. Add PJRC's package index, update, and install the core (we need **version ≥ 1.61.0**):

```
arduino-cli config init
arduino-cli config add board_manager.additional_urls https://www.pjrc.com/teensy/package_teensy_index.json
arduino-cli core update-index
arduino-cli core install teensy:avr
```

Check the version you got:

```
arduino-cli core list
```

You should see `teensy:avr` at **1.61.0** or newer.

> 💡 Prefer a GUI? You can instead install **Teensyduino 1.61** through the Arduino IDE 2.x Boards Manager (PJRC's package URL above) — that installs the same `teensy:avr` core. Either way, the build script just needs the core present and `arduino-cli` on your PATH.

---

6 · The libraries

The TŒRN firmware compiles against three **custom drivers** that adapt it to our cheap hardware. They are **already in the repo** under `firmware/libraries/`, and the build script points `arduino-cli` at them automatically — you don't install them:

Library	Replaces	Where the hardware config lives
i2cEncoderLibV2	TŒRN's 4 I²C RGB encoders → our 4 KY-040 mechanical encoders	`ICHOS_ENC_PINS` in `i2cEncoderLibV2.h`
FastTouch	TŒRN's 3 capacitive touch pads → our 3 momentary tact switches	`ICHOS_BTN_PINS` in `FastTouch.h`
IchosOled	adds the I²C **OLED HUD** (the state TŒRN showed on the encoder RGB rings)	`firmware/libraries/IchosOled/`

> 🧭 **Where the hardware lives:** pin assignments live in those two headers (`ICHOS_ENC_PINS` , `ICHOS_BTN_PINS`) and in `firmware/build_toern.py` — **not** in a `config.h` . (`config.h` belongs to the old NI404 fallback build and is not used by the TŒRN port.)

The build script also resolves any remaining stock libraries (the **Audio**, **Wire**, **SD**, etc. bundled in the Teensy core, plus anything in your user `Arduino/libraries/`) the way `arduino-cli` normally does.

---

7 · How the build script works

`firmware/build_toern.py` compiles the **unmodified** TŒRN sketch, applying only the minimal changes that make it run on our hardware. Step by step:

1. **Locate the sources.** It looks for the TŒRN sketch under `emulator/toern-src/` ; if it isn't there, it **clones it** from upstream automatically (so the repo stays free of a vendored copy).
2. **Stage them** into a temporary build folder (the originals stay pristine).
3. **Pin remap.** TŒRN's `SWITCH_1/2/3` default to pins that collide with our right-hand KY-040, so they are remapped to **25 / 26 / 28** — the same pins as `ICHOS_BTN_PINS` in `FastTouch.h` .
4. **Feature trim.** It removes the optional **2nd LED strip** (frees a pin and saves CPU), matching our build.

5. **OLED HUD.** It wires in the `IchosOled` library (include + `begin()` + a one-line render call in `loop()`) so the status HUD compiles as its own translation unit.
6. **Compile** with `arduino-cli` for the FQBN below and copy the result to a stable name, `toern.hex`.

The exact target it builds for is:

```
teensy:avr:teensy41:usb=serialmidi16,opt=o1std
```

- `usb=serialmidi16` → **USB type Serial + MIDIx16** (so `usbMIDI` works).
- `opt=o1std` → **`-O1`**. This is the critical flag: the default `-O2` crashes the Teensy gcc on TŒRN's giant single translation unit. `-O1` builds cleanly and the firmware still fits with room to spare.

---

8 · Building and uploading

8.1 Build

From the repo root:

```
python firmware/build_toern.py
```

When it finishes you'll have **`firmware/build/toern.hex`**.

8.2 Build and flash in one step

If you have `teensy_loader_cli` on your PATH:

```
python firmware/build_toern.py --flash
```

1. Connect the Teensy 4.1 via USB (a **data** cable).
2. The loader usually programs on its own; if it stays waiting, press the **little white button** on the Teensy.
3. When the flash finishes, the animation starts on the LED matrix.

8.3 Flashing the firmware

If you'd rather flash a **prebuilt** `.hex` (or don't have `teensy_loader_cli`), use the **one-click GUI flasher** in **`tools/flasher/`** — point it at `firmware/build/toern.hex` and click flash.

---

9 · Building from the Arduino IDE (alternative)

You can also build TŒRN by hand in the Arduino IDE 2.x — useful for stepping through the code. You'll need to reproduce what the script does:

Tools menu item	Value
Board	Teensy 4.1
USB Type	**Serial + MIDIx16**
CPU Speed	600 MHz (default)
Optimize	**"Fast"** ⚠️ (this is `-O1`; the default "Faster"/ `-O2` crashes the compiler)
Port	the Teensy's port (after connecting it)

You'll also have to apply the pin remap (`SWITCH_1/2/3` → 25/26/28), make the three custom libraries in `firmware/libraries/` visible to the IDE, and wire in the OLED HUD by hand. **The script does all of this for you**, which is why it's the recommended path.

> ⚠️ Don't forget **Optimize** → **"Fast"**. If you leave it at the default, the build dies with an internal compiler error (`exit 0xffffffff`) partway through TŒRN's single huge file.

---

10 · Optional tools (Python, Git)

10.1 Python

The build script needs **Python 3**. (Git is also required — see below — because the script clones the TŒRN sources on first run.)

Windows: `winget install Python.Python.3.12` · **macOS:** `brew install python@3.12`

> ℹ️ Any modern Python 3 works for the build script. (If you also use the `wavmaker` sample converter and run it from source on **Python 3.13+**, that one needs `pip install audioop-lts`, because `audioop` was removed from the standard library — but the ready-made `wavmaker.exe` in `sdcard/` needs no Python at all.)

10.2 Git and GitHub CLI

Git is required (the script clones TŒRN). GitHub CLI is optional.

Windows: `winget install Git.Git` (+ `GitHub.cli` optional) **macOS:** `xcode-select --install` (includes git) · `brew install gh` (optional)

---

11 · Checklist and common problems

✓	Check
☐	`arduino-cli version` works (it's on your PATH)
☐	`arduino-cli core list` shows **teensy:avr ≥ 1.61.0**
☐	**Python 3** and **Git** are installed
☐	`python firmware/build_toern.py` produces `firmware/build/toern.hex`
☐	(IDE build only) USB Type = Serial + MIDIx16 **and** Optimize = "Fast"
☐	Teensy 4.1 has **16 MB PSRAM** soldered

Symptom	Cause / fix
`arduino-cli not on PATH …` from the script	install arduino-cli (ch. 4) and reopen the terminal
Compile dies with `exit 0xffffffff` / internal compiler error	you built at `-O2`; use the script, or set IDE Optimize → "Fast" (`-O1`)
`teensy:avr` missing / too old	run `arduino-cli core install teensy:avr` and check `core list` (need ≥ 1.61.0)
Script can't clone the TŒRN sources	install Git, or clone manually into `emulator/toern-src/`
`'usbMIDI' was not declared` (IDE build)	USB Type not set to Serial + MIDIx16
`teensy_loader_cli not found` on `--flash`	use the GUI flasher in `tools/flasher/` instead
Teensy not programming	press the **white button** on the board; or use a **data** USB cable, not charge-only
Port not visible	charge-only USB cable → use a data cable

---

*Part of the **ichosynth** project · runs the real **TŒRN** (SP_) firmware · for the ICHOS 2026 workshop.*